

Backend Engineering Intern – Case Study

Candidate: Raman Tripathi

Case Study: StockFlow – B2B Inventory Management System

Overview

This document contains my responses to all three parts of the case study:

1. Code Review & Debugging
2. Database Design
3. API Implementation (Low Stock Alerts)

I've focused on:

- Correctness over shortcuts
- Real-world backend behavior (transactions, consistency, auditability)
- Clear assumptions where requirements were incomplete

Part 1: Code Review & Debugging

Issues Identified

1. No Transaction Safety

Two separate database commits are used:

- One for product creation
- One for inventory creation

If something goes wrong while creating the inventory (for example: invalid initial_quantity, DB constraint failure, etc.), the product is already committed.

2. Partial Data Risk

If inventory creation fails, the product still exists, leading to orphaned records(No Parent).

3. SKU Uniqueness Not Enforced

SKU must be unique across the platform, but the code does not check or enforce this.

4. No Input Validation

The code directly accesses `request.json['field']`, which can cause runtime errors if fields are missing.

5. Unsafe Price Handling

Price can be decimal, but there's no validation or type handling. Floating-point issues may occur.

6. Incorrect Assumption About Warehouses

Business logic says products can exist in multiple warehouses, but this endpoint assumes:

- One warehouse per product creation
- Inventory is always created immediately

7. No Warehouse Existence Check

it assumes `warehouse_id` is valid. If it's not, DB constraints may fail.

8.No handling of optional fields

Some fields might be optional, but code assumes all fields are always present.

Impact in Production (What can go wrong)

1.Impact of Issue 1 & 2 (Most critical)

- Product created without inventory
- Inventory dashboard shows wrong stock
- Business users lose trust in system

2.Impact of Issue 3 (SKU uniqueness)

- Duplicate SKUs break reporting
- Orders mapped to wrong products
- Revenue and analytics become unreliable

3.Impact of Issue 5 (Price)

- Rounding errors in billing
- Incorrect invoices
- Financial mismatch for B2B customers.

4.Impact of Issue 6 (Multiple warehouses)

- System becomes hard to extend
- Incorrect assumption that `product == warehouse`
- Future features become complex to add

5.Impact of Issue 7 (Warehouse check)

- Foreign key failures
- Runtime crashes in production.

Corrected Version (With Explanation)

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json or {}

    # Basic validation
    required_fields = ['name', 'sku', 'price']
    for field in required_fields:
        if field not in data:
            return {"error": f"{field} is required"}, 400

    try:
        price = Decimal(str(data['price']))
    except:
        return {"error": "Invalid price format"}, 400

    warehouse_id = data.get('warehouse_id')
    initial_quantity = data.get('initial_quantity', 0)

    try:
        with db.session.begin(): # Atomic transaction

            # Check SKU uniqueness
            existing = Product.query.filter_by(sku=data['sku']).first()
            if existing:
                return {"error": "SKU already exists"}, 409

            product = Product(
                name=data['name'],
                sku=data['sku'],
                price=price
            )
            db.session.add(product)
            db.session.flush() # Get product.id without committing

            # Inventory is optional (product may exist without stock initially)
            if warehouse_id is not None:
                inventory = Inventory(
                    product_id=product.id,
                    warehouse_id=warehouse_id,
                    quantity=initial_quantity
                )
                db.session.add(inventory)

            return {
                "message": "Product created successfully",
                "product_id": product.id
            }, 201

    except IntegrityError:
        db.session.rollback()
        return {"error": "Database constraint failed"}, 400

    except Exception as e:
        db.session.rollback()
        return {"error": "Internal server error"}, 500
```

Part 2: Database Design

I'm assuming a **multi-tenant B2B SaaS**, where data is scoped per company, but SKUs are globally unique as mentioned earlier.

1. Company

```
CREATE TABLE companies (  
  id UUID PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Why: All warehouses, products, and supplier relationships live under a company boundary.

2. Warehouse

```
CREATE TABLE warehouses (  
  id UUID PRIMARY KEY,  
  company_id UUID NOT NULL REFERENCES companies(id),  
  name VARCHAR(255) NOT NULL,  
  location TEXT,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_warehouses_company ON warehouses(company_id);
```

Why: A company can have many warehouses. Index helps list warehouses per company quickly

3. Product

```
CREATE TABLE products (  
  id UUID PRIMARY KEY,  
  company_id UUID NOT NULL REFERENCES companies(id),  
  name VARCHAR(255) NOT NULL,  
  sku VARCHAR(100) NOT NULL UNIQUE,  
  price DECIMAL(10, 2) NOT NULL,  
  is_bundle BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_products_company ON products(company_id);
```

Why:

- SKU must be unique platform-wide → enforced at DB level
- DECIMAL for price avoids floating-point errors
- `is_bundle` simplifies querying bundle vs normal products

4.Inventory (Current Stock)

id (PK)

product_id (FK)

warehouse_id (FK)

quantity

UNIQUE (product_id, warehouse_id)

5.Inventory Change Log (history)

- id (PK)
- product_id (FK)
- warehouse_id (FK)
- quantity_delta
- previous_quantity
- new_quantity
- change_source
- created_at

6.Suppliers

- id (PK)
- name
- contact_info

7.SupplierProducts (many-to-many)

- supplier_id (FK)
- product_id (FK)
- cost_price
- lead_time_days

. Design Decisions & Justifications

Why separate Inventory & InventoryChangeLog?

Why enforce constraints at DB level?

- SKU uniqueness
- Product-warehouse uniqueness

Because app-level checks fail under concurrency.

Why indexes on foreign keys?

- Most queries will be:
"All inventory for a warehouse"

2. Gaps & Questions for Product Team

These are things I'd clarify before finalizing:

1. Are **products global** or always company-scoped?
2. Can **suppliers serve multiple companies**, or are they company-specific?
3. Can a **bundle contain another bundle**, or only simple products?
4. Should inventory changes track **who made the change** (user / system)?
5. Do we allow **negative inventory** (backorders)?
6. Do we need **soft deletes** for compliance/auditing?
7. Should SKUs be **case-sensitive** or normalized?

Part 3: API Implementation – Low Stock Alerts

EndPoint

GET /api/companies/{company_id}/alerts/low-stock

Assumptions (documented clearly)

Product type determines threshold

- Stored as product_type on products table
- Thresholds stored in a config table or computed via mapping

Recent sales activity

- A product is considered “active” if it had at least one sale in the last 30 days
- Sales data exists in a sales table

Supplier selection

- Each product has at least one supplier
- For simplicity, we return the primary supplier (lowest cost or first mapped)

Days until stockout

- Calculated using average daily sales from recent history
- If avg sales = 0 → days_until_stockout = null

Code:

```
@app.route('/api/companies/<company_id>/alerts/low-stock', methods=['GET'])
def low_stock_alerts(company_id):
    alerts = []
```

```
# Assume thresholds based on product type
```

```
PRODUCT_THRESHOLDS = {
    "FAST_MOVING": 20,
    "NORMAL": 10,
    "SLOW_MOVING": 5
}
```

```
# Define recent activity window
```

```
recent_window = datetime.utcnow() - timedelta(days=30)
```

```
# Step 1: Fetch active products with recent sales
```

```

active_products = (
    db.session.query(
        Product.id,
        Product.name,
        Product.sku,
        Product.product_type
    )
    .join(Sale, Sale.product_id == Product.id)
    .filter(
        Product.company_id == company_id,
        Sale.created_at >= recent_window
    )
    .distinct()
    .all()
)

for product in active_products:
    threshold = PRODUCT_THRESHOLDS.get(product.product_type, 10)

    # Step 2: Fetch inventory per warehouse
    inventories = (
        db.session.query(
            Inventory,
            Warehouse.name.label("warehouse_name")
        )
        .join(Warehouse, Warehouse.id == Inventory.warehouse_id)
        .filter(Inventory.product_id == product.id)
        .all()
    )

    for inventory, warehouse_name in inventories:
        if inventory.quantity >= threshold:
            continue # Not low stock

    # Step 3: Calculate avg daily sales
    avg_daily_sales = (
        db.session.query(func.avg(Sale.quantity))
        .filter(
            Sale.product_id == product.id,
            Sale.created_at >= recent_window
        )
        .scalar()
    )

    days_until_stockout = None
    if avg_daily_sales and avg_daily_sales > 0:
        days_until_stockout = int(inventory.quantity / avg_daily_sales)

    # Step 4: Fetch supplier info
    supplier = (
        db.session.query(Supplier)
        .join(SupplierProduct)
        .filter(SupplierProduct.product_id == product.id)
        .first()
    )

    alerts.append({
        "product_id": product.id,

```

```

        "product_name": product.name,
        "sku": product.sku,
        "warehouse_id": inventory.warehouse_id,
        "warehouse_name": warehouse_name,
        "current_stock": inventory.quantity,
        "threshold": threshold,
        "days_until_stockout": days_until_stockout,
        "supplier": {
            "id": supplier.id if supplier else None,
            "name": supplier.name if supplier else None,
            "contact_email": supplier.contact_email if supplier else None
        }
    })

return jsonify({
    "alerts": alerts,
    "total_alerts": len(alerts)
}), 200

```

Explanation:

1. Filter only relevant products
 - Belonging to the company
 - With sales activity in last 30 days
2. Evaluate inventory per warehouse
 - Because stock is warehouse-specific
 - One product can generate multiple alerts
3. Apply dynamic thresholds
 - Based on product type
 - Allows business flexibility
4. Estimate stockout time
 - Based on recent average sales
 - Helps businesses prioritize reordering
5. Attach supplier info
 - Makes alert actionable
 - Reduces friction for reordering

Edge Cases Handled:

1. No recent sales
 - Product is skipped → avoids noisy alerts
2. Zero average sales
 - days_until_stockout = null (safe handling)
3. Multiple warehouses
 - Alerts generated per warehouse
4. Missing supplier
 - Supplier fields returned as null, API doesn't fail
5. Unknown product type
 - Default threshold applied

